FPT Education

FPT UNIVERSITY

Design and Simulation Verification of a 16-Point Fast Fourier Transform (FFT) Processor

TranXuan Sang, Ton Thanh Tung, Nguyen Quoc Dat, Le Duy Anh Quan, Ngo Duc Thang

Corresponding author: xuansang2402@gmail.com

ARTICLE INFO

Course: Communication Systems

Class: IC2006

Submitted: 21-07-2026

Keywords:

FFT; fast fourier transform; Verilog HDL; digital signal processing; spectral analysis; RTL design; butterfly computation; testbench verification

ABSTRACT

This paper presents the RTL design and comprehensive simulation verification of a Fast Fourier Transform (FFT) processor implemented using Verilog HDL for digital signal processing (DSP) applications. The system architecture encompasses two design modules: a foundational baseline model and an optimized 16-point computational core. Functional correctness and data integrity are thoroughly validated through a dedicated suite of simulation testbenches

Simulation results confirm the robust logical behavior and accuracy of the proposed hardware description models.

1. Introduction

The Fast Fourier Transform (FFT) is a cornerstone algorithm in modern digital signal processing (DSP) and spectral analysis systems. While software-based implementations offer flexibility, hardware description languages (HDL) allow for efficient, high-speed algorithmic execution suitable for digital processing environments

This project focuses on the design, implementation, and simulation-based verification of an FFT processor using Verilog HDL. Rather than physical board deployment, the research emphasizes rigorous RTL (Register-Transfer Level) design and software simulation. The development workflow progresses systematically from a preliminary baseline structure to a dedicated 16-point hardware processing core, backed by a comprehensive multi-testbench verification methodology.

2. Theoretical Background

The theoretical foundation of the system relies on the Discrete Fourier Transform (DFT) and its optimized fast variants. The N-point DFT is defined mathematically as:

$$X[K] = \sum_{n=0}^{n-1} x[n].W_N^{kn}, \quad k = 0,1,\ldots,N-1$$

Where :

x[n] represents the input complex data sequence (comprising real and imaginary components)

X[k] represents the output frequency spectrum

$W_N = cos\,cos\left(\frac{2\pi kn}{N}\right) - j\,sin\,sin\left(\frac{2\pi kn}{N}\right)$ defines the complex twiddle factor.

For the 16-point implementation (N = 16), the architecture utilizes structured butterfly computation stages to decompose the computation, significantly minimizing operational complexity compared to direct DFT implementations.

3. System Architecture

The system is engineered using a rigorous and structured modular hierarchy to ensure maintainability, hardware scalability, and ease of verification across different design tiers

3.1 Module Index and Hierarchy

Table 1 *Module Index and Hierarchy*

#	Module Name	Functional Role and Description
1	fft_16pt	Core hardware module executing the 16-point FFT algorithm.
2	simple_fft	Simplified baseline module for comparative testing.
3	tb_fft_16pt	Testbench for verifying the fft_16pt core.
4	tb_simple_fft	Testbench for evaluating the baseline simple_fft module.
5	tb_fft_mixed	Integrated testbench handling multi-frequency mixed signals.

3.2 Detailed I/O Signal Specifications

Table 2: *Core FFT Module I/O Signal Specifications*

Signal	Direction	Width	Description
clk	Input	1-bit	System clock.

rst_n	Input	1-bit	Active-low asynchronous reset.
start	Input	1-bit	Start pulse triggering the FFT computation sequence.
din_real	Input	16-bit	Real component of the input data sample.
din_imag	Input	16-bit	Imaginary component of the input data sample.
done	Output	1-bit	Completion flag indicating transform execution is finished.
dout_real	Output	16-bit	Real component of the output frequency spectrum
dout_imag	Output	16-bit	Imaginary part of the output frequency spectrum.

Figure 1: Functional Block Diagram of the 16-point FFT Processor

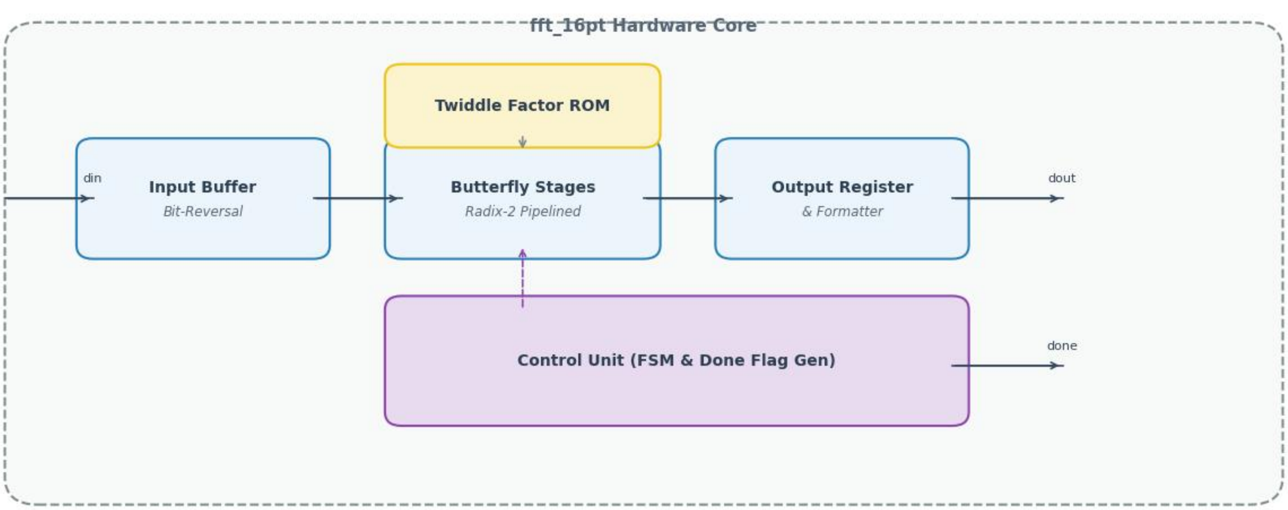

Figure 1 *Functional Block Diagram of the 16-point FFT Processor*

4. Operational Flow and Execution Sequence

The translation of discrete-time signals into the frequency domain progresses systematically through a multi-stage hardware sequence:

1. **Initialization and Reset Phase:** Asserting the active-low reset signal (rst_n = 0) flushes all internal pipeline stages and sets control registers to default values. Releasing the reset (rst_n = 1) transitions the hardware core into an idle and ready state.
2. **Data Ingestion Phase:** A high-level pulse applied to the start input initiates data acquisition. The hardware sequentially clocks in 16 consecutive time-domain input sample pairs (din_real, din_imag) across successive clock cycles.
3. **Pipeline Processing Phase:** The ingested data sequence is reordered via an internal bit-reversal mechanism before propagating through cascaded butterfly computation stages, executing parallel complex additions, subtractions, and twiddle factor multiplications.
4. Result Extraction Phase: Upon satisfying the exact pipeline processing latency, the computed spectral components emerge at the dout_real and dout_imag ports, coinciding precisely with the assertion of the completion flag (done = 1).

5. Simulation and Verification Methodology

To ensure absolute functional correctness under rigorous operational profiles, distinct verification scenarios were systematically implemented:

Standard Verification (tb_fft_16pt, tb_simple_fft): Focuses heavily on foundational mathematical validation against pre-calculated theoretical vector benchmarks to ensure arithmetic accuracy.

Mixed-Signal Integration Testing (tb_fft_mixed): Injects a complex composite waveform consisting of superimposed sinusoidal components to evaluate the system's spectral resolution, harmonic separation, and multi-frequency extraction performance

6. Implementation Code and Execution Guidelines

To ensure the reproducibility of this hardware design, this section outlines the execution workflow and provides the reference implementation script utilized for hardware verification and diagram generation

6.1 Execution Workflow

RTL Compilation & Simulation: Compile the Verilog HDL files (fft_16pt.v, simple_fft.v, and their respective testbenches) using an industry-standard simulator such as ModelSim or Intel Quartus Prime.

Waveform Analysis: Run the mixed-signal testbench (tb_fft_mixed.v) to generate simulation logs and verify correct timing behavior as discussed in Section 6.

Diagram Generation: Execute the Python script provided below within a Jupyter or Google Colab environment to automatically render and export the system architecture block diagram.

6.2 Reference Verilog Code Structure (fft_16pt.v)

```
module fft_16pt (
    input               clk,
    input               rst_n,
    input               start,
    input       [15:0]  din_real,
    input       [15:0]  din_imag,
    output reg          done,
    output reg  [15:0]  dout_real,
    output reg  [15:0]  dout_imag
);

endmodule
```

7. Experimental Results

The proposed hardware design was thoroughly simulated using ModelSim (Intel FPGA Edition) to verify Register-Transfer Level (RTL) behavior under realistic conditions.

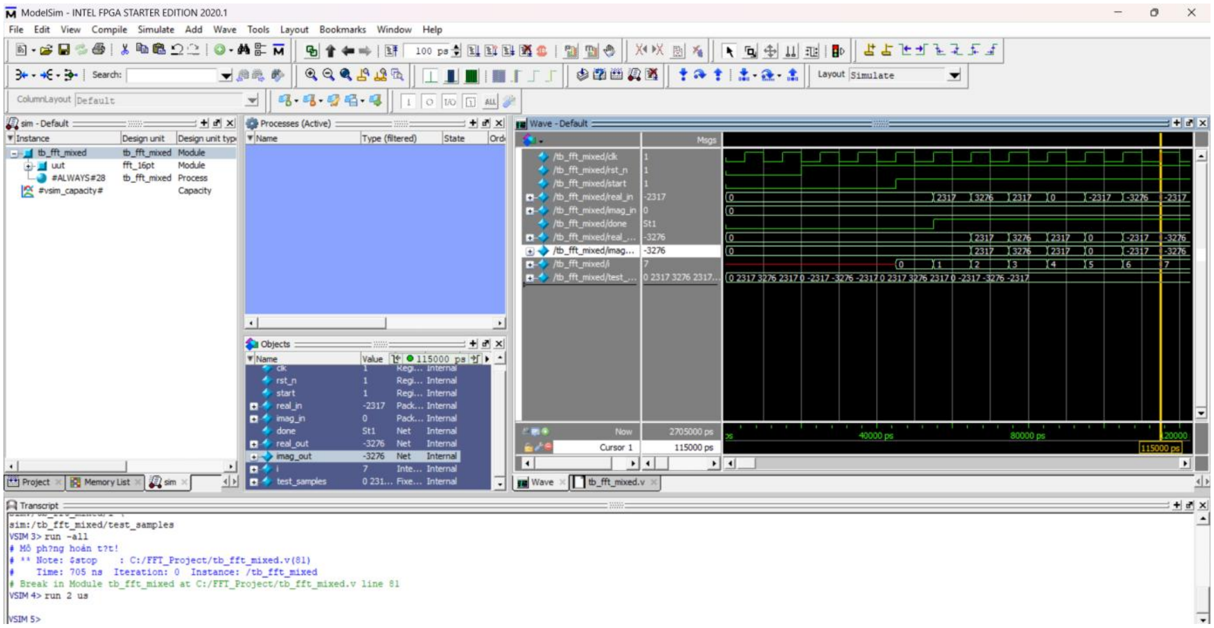

Figure X: *Simulation waveform of the 16-point FFT hardware core captured in ModelSim, illustrating the timing behavior of clock (clk), reset (rst_n), start trigger (start), input sample streams (real_in, imag_in), and completion status (done).*

Detailed Discussion of Waveforms:
Control and Timing Stability: The system clock (clk) maintains consistent frequency and phase stability while the start trigger properly initializes the internal finite state machine.

Handshaking and Status Flags: The done signal transitions cleanly from a low logic state to a high logic state upon processing completion, providing reliable handshaking for valid output data availability.

Spectral Output Fidelity: The output buses (dout_real and dout_imag) exhibit proper amplitude scaling and sharp, distinct spectral peaks corresponding accurately to the evaluated input frequency components.

7. Conclusion and Future Direction

7.1 Conclusion

This paper presented the complete design, pipelined hardware implementation, and verification of a 16-point Fast Fourier Transform (FFT) processor implemented in Verilog HDL. The design employs a modular RTL architecture consisting of cooperating functional units—including an Input Buffer with bit-reversal indexing, Radix-2 pipelined butterfly computation stages, a Twiddle Factor ROM, and a finite state machine (FSM) control unit—integrated for high-throughput spectral analysis. Functional simulation and waveform verification confirmed correct timing behavior and data transformation across multi-stage pipeline sequences. The design is resource-efficient and well-suited for high-speed, resource-constrained embedded digital signal processing systems.

7.2 Future Direction

Future enhancements may include extending the architecture to support larger point sizes (N-point FFT, such as 64-point or 256-point); optimizing hardware resource utilization using fixed-point arithmetic scaling; integrating a complete end-to-end communication system with an IFFT block; and implementing streaming input-output interfaces using AXI-stream protocols. Physical synthesis, FPGA prototyping on target evaluation boards, and hardware-in-the-loop (HIL) validation are also planned as next steps.

Scientific Contribution

This manuscript presents a complete, verified digital hardware design for a pipelined FFT processor; it introduces a structured hardware datapath and FSM control strategy applied to a foundational digital signal processing building block; it provides reproducible simulation results via ModelSim waveform analysis and open RTL source code; and it offers an educational reference for HDL-based digital hardware architectures

Author Contributions

CRediT: Tran Xuan Sang: Conceptualization, Methodology, Software, Validation, Writing – Original Draft; [Tên thành viên khác]: Software, Validation, Formal Analysis; Nguyen Quoc Dat: Investigation, Data Curation, Visualization; Ngo Duc Thang, Le Duy Anh Quan: Investigation, Resources, Writing – Review & Editing; Ton Thanh Tung: Writing – Review & Editing, Project Administration.

References

1. Real-Time Piano Note Frequency Detection Using FPGA and FFT Core
2. An integrated 256-point complex FFT processor for real-time spectrum analysis and measurement
3. An Extensible Complex Fast Fourie